Formal Proofs and Algorithmic Analysis of Braun Trees in Coq

Filiuta Alexandru

February 11, 2025

Abstract

Braun Trees are balanced binary trees that provide efficient implementation options for extensible arrays and priority queues, guaranteeing logarithmic performance. Despite their utility, only limited formal verification solutions have been applied to Braun Trees. This thesis addresses that gap by specifying Braun Trees in Coq, developing an invariant that preserves their balanced structure, and formally proving the correctness of several useful operations: size, replicate, lookup, update, insert, remove and tree to list conversion. In addition to standard algebraic specifications, we leverage a model-based approach by relating Braun Trees to Coq's built-in lists. We prove that each operation maintains the Braun Tree invariant, and we validate key properties such as preserving size or maintaining order. Our results offer proofs for Braun Trees's reliability and correctness.

1 Introduction

1.1 Motivation

Formal verification uses rigorous mathematical proofs to ensure that software behaves exactly as intended under all specified conditions. In a world where even small errors in data structures can lead to major failures such reassuring methods become vital. This thesis focuses on **Braun Trees**, a balanced data structure with a strict and definitory shape property: each node's left subtree is the same size or one larger than its right subtree. This balanced shape yields efficient, logarithmic-time performance for key operations.

Despite such advantage, fully formal proofs on Braun Trees remain scarce. A principal challenge is that the shape-based property must be carried and maintained globally. In addition, certain algorithms require sophisticated termination arguments, which further complicate the verification attempts. By formally specifying Braun Trees and proving the correctness of their fundamental operations in Coq, this thesis helps closing the gap in academic research and reinforces the benefits of formal methods in reasoning about algorithms and data structures.

1.2 Approach

The general approach of this thesis begins by defining the Braun Trees along with a shape-based invariant. Once the type is defined, we identify and formally describe the main operations of this project together with the behavior and properties they must satisfy. To guarantee the tree remains balanced at all times, every function must be shown to output a valid Braun Tree whenever given a valid Braun Tree as input. Hence, proving the operations preserve the invariant is the foundational requirement for our work. After establishing the **invariant maintenance**, we adopt two distinct **specification** approaches:

1. **Algebraic Specifications**: we specify each operation in terms of core algebraic properties. For example, consider inserting an element x into a Braun Tree t. The property

$$\texttt{sizeOrg (insert x t)} = \texttt{sizeOrg (t)} + 1$$

states that inserting x strictly increases the number of nodes in t by exactly one. Here, `sizeOrg` is the function that returns the total number of nodes in the tree. Such algebraic equations serve as a high-level correctness specification, which we then prove using Coq.

2. **Model-Based Specifications**: we analyze the correctness of Braun Trees in relation to a simpler structure already present in the Coq standard library, the lists. The conversion from trees to lists is made by the function *tree_to_list*, which facilitates the correspondence between the tree operations and the already implemented list operations. This "model-based" technique offers a clear reference model, making the correctness arguments more intuitive and cross-verifying the algebraic laws against a well-known abstraction.

1.3 Problem Statement

Our project addresses the need for a rigorous, machine-verified framework for Braun Trees. So far, no extensive formal verification has confirmed that all general operations preserve the balance property and adhere to the expected behavior. In this regard, the main questions proposed are:

1. Can we define in a formal system each fundamental operations and prove that they maintain the shape of the Braun Tree and produce consistent results?
2. Do the approaches used capture the correctness of the intended behavior?
3. By leveraging the shape property of Braun Trees, can we formally define logarithmic time complexity for the majority of the operations?

By addressing these questions, this thesis demonstrates the reliability of Braun Trees along with their practical use in priority queues, flexible arrays, and other data structures alike.

1.4 Related Work

Initially, Braun Trees were introduced by Braun and Rem and used by Hoogerwoord as an efficient structure for flexible arrays [2]. Chris Okasaki introduced several logarithmic-time algorithms on them [3]. Prior formalizations have mostly appeared in Why3 and Isabelle/HOL and were done by Nipkow and Sewell [4]. While formalization in Coq has been discussed by Filliâtre, his focus was on general binary trees [5]. Our work builds on those previous papers, providing not only correctness proofs but also time-complexity specifications for all core operations.

1.5 Coq

Coq is an interactive theorem proving system based on the Calculus of Inductive Constructions, derived from Thierry Coquand's Calculus of Constructions. At its core it serves both as a higher-order logic language, enabling quantification over predicates and functions, and as a functional language (also called Gallina), which provides a strong logical foundation: every function must terminate. This blend is described as a "bridge between programming and logic" by the Software Foundations series [6]. In practice, this means a Coq development can mix function definitions, statements of theorems, and interactive proofs in one file, guaranteeing that every part of executable code is accompanied by precise logical specifications and machine-checked proofs. Coq's key features include:

- letting users guide Coq with "tactics" in a step-by-step manner, generating subgoals until a proof term is fully validated.
- automation, since Coq's standard library of tactics (e.g., auto, lia, etc...) can be employed to dismiss many logically obvious proof obligations.
- extraction mechanisms that allows verified functions to be extracted to mainstream languages like OCaml or Haskell.

1.6 Outline

The remainder of this thesis is organized as follows. First, we present the data structure of Braun Trees, beginning with their type definition and their characteristic shape invariant. We then introduce the operations defined on these trees showcasing both their implementations and the formal proofs. Afterwards, we provide a complexity analysis, showing how these operations achieve logarithmic running time, followed by our key results. Finally, we discuss the conclusions and potential future works, followed by acknowledgments.

2 Braun Trees

2.1 What are Braun Trees

Braun Trees are a special form of balanced binary trees in which each node's left subtree is either the same size as its right subtree or exactly one element larger. Concretely, if l and r denote the left and right subtrees, and $|l|$ and $|r|$ are their sizes (*i.e.* the number of nodes), then:

$$|l| \;=\; |r| \quad \vee \quad |l| \;=\; |r| + 1. \tag{1}$$

This shape property is applied recursively down the entire tree. Figure 1 illustrates a Braun Tree of size 15, with nodes indexed by their level-order positions. Because of the structural balance, the **height** of a Braun Tree with n nodes is always $\Theta(\log n)$. Moreover, the shape constraint holds *recursively* for every node in the tree, propagating from the root, as we go down to the leaves. As a result, there can be exactly one tree shape that satisfies this requirement for a given number of nodes n.

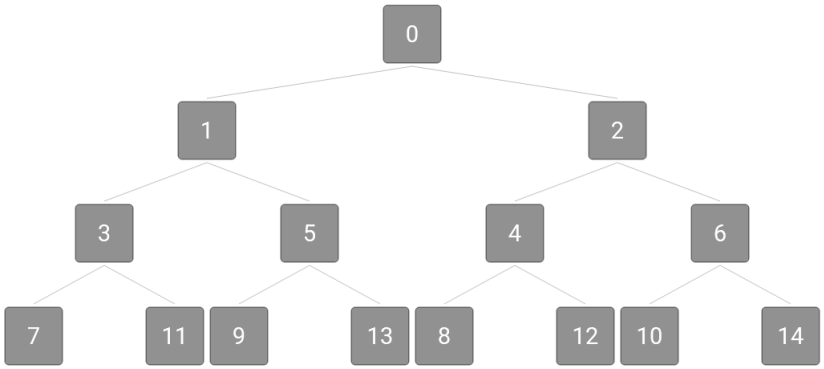

Figure 1: A Braun Tree of size 15

A practical consequence of this uniform shape is that the nodes in the left subtree of the root correspond to *odd* indices, while nodes in the right subtree correspond to *even* indices. This property is often used for **dynamic arrays**, where each node represents a position **n** in the array, and moving left or right is determined by checking the parity of **n** and then halving it. Hence, both **lookup** and **update** take $O(\log n)$ time when implemented using Braun Trees.

Braun Trees also serve as an efficient **priority queue** representation. When used as a heap, a Braun Tree's balanced shape ensures that an insert adds a new element in such a way that the size condition remains satisfied. Since only one path from root to leaf is adjusted, the time cost is $O(\log n)$. On the other hand, removing the highest-priority element also takes $O(\log n)$ time, by using a partial swapping strategy. Thus, Braun Trees provide efficient choices for both **array-like** indexing and **heap-like** operations.

2.2 Type Definition

We define the Braun Tree inductive type in Coq as follows:

```
Inductive BraunTree (V : Type) : Type :=
  | Empty
  | Braun (l : BraunTree V) (v : V) (r : BraunTree V).
```

Listing 1: Braun Trees definition

A Braun Tree over any type **V** can be either empty:

(**Empty**), base case

or made up of a node with a left Braun subtree, a value, and a right Braun subtree:

(**Braun l v r**), recursive case

The size constraint is not enforced by the type itself but rather by a separate invariant predicate presented below.

2.3 The Invariant

An invariant is a property that a data structure must satisfy at any stage. In the case of Braun Trees the invariant is represented by the balanced size property. For capturing the invariant, we define:

1. A size function *sizeOrg*, that calculates the number of Nodes in the tree:

```
Fixpoint sizeOrg {V : Type} (t : BraunTree V) : nat :=
    match t with
    | Empty ⇒ 0
    | Braun l _ r ⇒ 1 + sizeOrg l + sizeOrg r
    end.
```

Listing 2: Size operation definition

2. An inductive predicate *IsBraun*, that ensures the balance condition is not violated:

```
Inductive IsBraun {V : Type} : BraunTree V -> Prop :=
| IsBraun_Empty :
    IsBraun Empty
| IsBraun_Node : forall l v r,
    IsBraun l ->
    IsBraun r ->
    (sizeOrg l = sizeOrg r \/ sizeOrg l = sizeOrg r + 1) ->
    IsBraun (Braun l v r).
```

Listing 3: Invariant definition

As **Listing 3** shows, **IsBraun** states that for every node, the size of its left subtree, *sizeOrg l*, differs from the size of its right subtree, *sizeOrg r*, by at most 1. Consequently, in a *level-order* labeling, the upcoming insertion lands in the left subtree whenever *sizeOrg l* = *sizeOrg r* (or, in other words, *sizeOrg t* = 2 * k), giving rise to the odd/even property. The invariant is absolutely necessary in proving the following algorithms due to the fact that the inductive constructor does not guarantee the size balance on the original tree. In Coq, we apply structural induction on *IsBraun* to prove that each operation maintains the invariant. If the premise implies that the input tree is a Braun Tree, then the resulting tree will also satisfy the Braun Tree condition.

3 Operations

3.1 Size

The first function implemented, represented by **sizeOrg**, was already shown in the Invariant section and computes the number of nodes in a tree, by doing a sequential traversal. Its time complexity is $O(n)$ because it inspects every single node. It does not exploit the shape's parity properties, which can yield faster methods. One such efficient size algorithm was proposed by Okasaki, which we defined as follows:

```
Fixpoint diff {V: Type} (s: BraunTree V) (m: nat) : nat :=
  match s, m with
  | Empty, 0 ⇒ 0
  | Braun _ _ _, 0 ⇒ 1
  | Braun l _ r, S (S m') ⇒
      if even m then diff r (div2 m)
      else diff l (div2 m')
  | _, _ ⇒ 0
  end.

Fixpoint size {V : Type} (t : BraunTree V) : nat :=
  match t with
  | Empty ⇒ 0
  | Braun l _ r ⇒ let m := size r in 1 + 2 * m + diff l m
  end.
```

Listing 4: Okasaki's size function with helper function diff

Here, the size function relies on a helper lemma, **diff**, that makes use of the Braun property in determining if the left subtree is "one step ahead" or not. **size** runs in $O(\log^2 n)$ rather than $O(n)$, because each call to **diff** descends roughly one level in the tree.

3.2 Replicate

The **replicate** operation builds a Braun Tree of a specified size n, where all nodes hold the same value x. The result of *replicate 1 3* can be seen in figure 2:

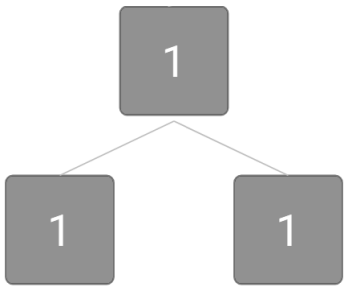

Figure 2: replicate 1 3

Two different approaches were taken into consideration when replicating an element in a Braun Tree:

1. A simple function with the time complexity $O(n \log n)$.
2. Chris Okasaki's improved, efficient, but more complex algorithm that runs in $O(\log n)$.

The first approach relies on repeating the **insert** function, which we will discuss later on, for all n elements:

```
Fixpoint replicate {V : Type} (x : V) (n : nat) : BraunTree V :=
  match n with
  | 0 ⇒ Empty
  | S n' ⇒ insert x (replicate x n')
  end.
```

Listing 5: Simple replicate

This recursive function is straightforward in its approach. It applies pattern matching on **n**, where each recursive step explicitly decreases the value of **n** by one, thus ensuring termination.

Okasaki provides a faster method that leverages the balanced shape to build two trees of size n and $n+1$ at the same time. We implemented this in Coq using a **divide and conquer** strategy:

```
Program Fixpoint copyOkasaki {V : Type} (x : V) (n : nat) {measure n} :
    (BraunTree V * BraunTree V) :=
  match n with
  | 0 ⇒ (Braun Empty x Empty, Empty)
  | S _ ⇒
    let (s, t) := copyOkasaki x (div2 (n - 1)) in
    if even n then (Braun s x s, Braun s x t)
              else (Braun s x t, Braun t x t)
  end.
Next Obligation.
  destruct wildcard' .
  - simpl. lia.
  - destruct wildcard' eqn:Hm.
    -- simpl. lia.
    -- apply Nat.lt_succ_r. apply Nat.lt_succ_r. Search div2. apply
       Nat.div2_decr. lia.
Qed.

Definition copyOkasakiComplete {V : Type} (x : V) (n : nat) : BraunTree V
    :=
  snd (copyOkasaki x n).
```

Listing 6: Okasaki's fast replicate implementation using a helper lemma and measure on the decreasing argument

The reason why in our case the slower definition was preferred is because Okasaki's replicate, while more efficient, does not use a straightforward structural recursion. Instead, it divides **n** approximately in half each step, which is not recognized by the proof system as a termination guarantee. To address this issue, we marked the decreasing argument with a **measure** annotation that explicitly states the recursion measure. To formally satisfy the termination requirement, Coq uses a feature called **Program Fixpoint** along with **Next Obligation**, that requires a formal proof to ensure the decrease of the given measure.

3.3 Insert

Braun Trees are particularly useful in the context of priority queues. The **insert** function in a Braun Tree acts similarly to the **enqueue** operation in a priority queue, where elements are added in a manner that preserves the priority. In the case of the Braun Trees, the new elements are added to the end of the tree structure, ensuring that the tree's structure is not violated. As an example, the result of *insert 2 t*, where t is the tree from figure 2, can be seen in the figure below:

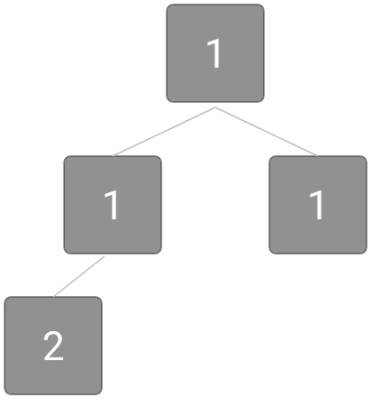

Figure 3: insert 2 (replicate 1 3)

```
Fixpoint insert {V : Type} (v : V) (t : BraunTree V) : BraunTree V :=
  match t with
  | Empty => Braun Empty v Empty
  | Braun l v' r =>
```

```
    if Nat.eqb (sizeOrg l) (sizeOrg r)
    then Braun (insert v l) v' r
    else Braun l v' (insert v r)
  end.
```

Listing 7: insert function

The implementation runs in $O(\log n)$ time, it assumes the tree involved satisfies the invariant and is using the size property in adding new elements:

- If both the left and right subtrees are of the same size, the new element **v** is inserted into the left subtree.
- If the left subtree is larger, the insertion occurs in the right subtree.

3.4 Remove

Complementary to **insert**, **remove** corresponds to the **dequeue** operation in priority queues, by removing the highest-priority element, the root. Applying remove on the tree from figure 3 will result in the tree below:

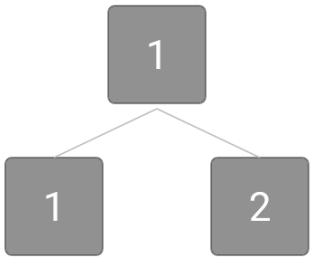

Figure 4: remove (insert 2 (replicate 1 3))

The removal process takes the indexing property into consideration and involves a swap between the left and right subtrees, in order to maintain the structural balance. **Remove** uses a helper lemma **removeRoot**, which outputs a tuple consisting of the deleted element and the new tree:

```
Fixpoint removeRoot {V : Type} (t : BraunTree V) : option (V * BraunTree
    V) :=
  match t with
  | Empty ⇒ None
  | Braun Empty v Empty ⇒ Some (v, Empty)
  | Braun l v r ⇒
      match removeRoot l with
      | Some (lv, newL) ⇒ Some (v, Braun r lv newL)
      | None ⇒ None (* unreachable if Braun property holds *)
      end
  end.

Definition remove {V : Type} (t : BraunTree V) : option (BraunTree V) :=
  match removeRoot t with
  | Some (_ , newT) ⇒ Some newT
  | None ⇒ None
  end.
```

Listing 8: remove and removeRoot definitions

This implementation eliminates the root and reconstructs the tree as follows:

1. It identifies the root of the left subtree and appends it onto the main root of the tree
2. To tree is reassembled by switching the left and right subtree, which ensures that the left subtree is equal to or larger than the right one.
3. The remove operation operates recursively in the right subtree until it reaches the end.

This approach represents the reason as to why Braun Trees are preferred for priority queues or heaps. It enables maintaining a balanced tree while ensuring that each removal operation also runs in $O(\log n)$ time, similar to the insertion. This consistency in performance, especially during structural modifications, is crucial for the efficiency of priority queues. Figure 5 illustrates how the algorithm preserves the Braun Tree structure.

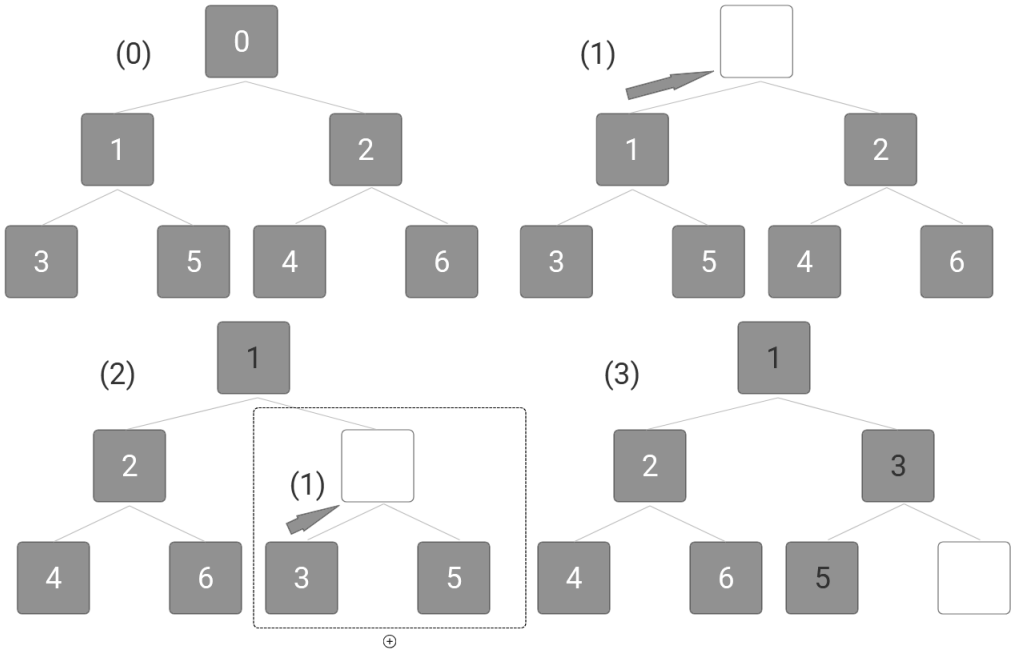

Figure 5: Removal steps in Braun Trees

3.5 Lookup

The **lookup** function provides an efficient way to access elements at a specified index **n**, making it a suitable match for the implementation of dynamic arrays where fast access is necessary. Performing *lookup 2 t*, where t is the tree from figure 4(*i.e.* remove (insert 2 (replicate 1 3))), will output the natural number 2. The function leverages the index parity property mentioned previously to navigate through the nodes of the tree.

```
Fixpoint lookup {V : Type} (t : BraunTree V) (n : nat) : option V :=
  match t with
  | Empty ⇒ None
  | Braun l v r ⇒
      match n with
      | 0 ⇒ Some v
      | S m ⇒ if Nat.even m
                then lookup l (Nat.div2 m)
                else lookup r (Nat.div2 m)
      end
  end.
```

Listing 9: lookup function definition

By virtue of Braun Trees' characteristic structure, all odd indices end up lying in the left subtree, while the even ones are placed in the right subtree. Therefore, the function decides recursively whether to move left or right by checking the parity of the index and reduces the search space by half with each call. This effectively leads to a logarithmic time complexity of $O(\log n)$, making it comparable with linear search methods.

3.6 Update

Similarly to **lookup**, the **update** function uses the index's parity in order to navigate the tree in a recursive and efficient way. It modifies the value of a given node while preserving the tree's structural balance. **Update** is crucial for dynamic arrays as well, offering a logarithmic time complexity of $O(\log n)$, to a data structure upon which changes to elements happen continually. The process of *update 2 3* on the tree from figure 4 can be seen below:

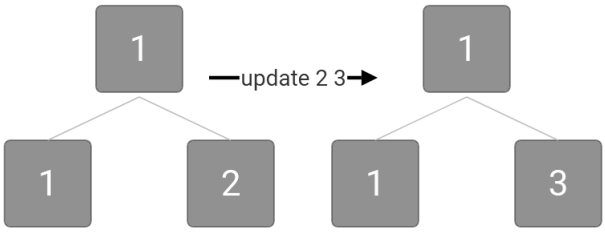

Figure 6: update 2 3 (remove (insert 2 (replicate 1 3)))

```
Fixpoint update {V : Type} (n : nat) (v : V) (t : BraunTree V) : BraunTree
    V :=
  match t with
  | Empty ⇒ Empty
  | Braun l v' r ⇒
      match n with
      | 0 ⇒ Braun l v r
      | S m ⇒
          if Nat.odd n
          then Braun (update (Nat.div2 m) v l) v' r
          else Braun l v' (update (Nat.div2 (n - 1)) v r)
      end
  end.
```

Listing 10: update function definition

3.7 Tree to List

The **tree_to_list** operation is the last one defined in our workload. It recursively traverses the Braun Tree, concatenating each node's value into a list. The order of traversal governed by the helper function **merge_lists** ensures that the elements are added to the list in a manner reflecting their original positions in the tree, with the root at the beginning followed by elements alternating from the left and right subtrees. Applying **tree_to_list** to the new tree from figure 6, will yield the list: 1; 1; 3.

Tree_to_list is essential for integrating Braun Trees into a broader context of data structures, and for enabling model-based specifications in their formal verification. By converting the abstract Braun Tree into a well recognized model such as the list, we can directly compare and relate the tree's operations to operations on lists, which are often better documented and verified.

```
Fixpoint merge_lists {V : Type} (l1 l2 : list V) : list V :=
  match l1, l2 with
  | [], _ => l2
  | _, [] => l1
  | x::xs, y::ys => x :: y :: (merge_lists xs ys)
  end.

Fixpoint tree_to_list {V : Type} (t : BraunTree V) : list V :=
  match t with
  | Empty => []
  | Braun l v r => v :: (merge_lists (tree_to_list l) (tree_to_list r))
  end.
```

Listing 11: tree_to_list conversion function along with the helper function for list merging

3.8 Proofs

3.8.1 Algebraic Specification

Algebraic specifications define the logical relationships between operations on Braun Trees. Most of the proofs involved in this project followed an algebraic approach. Some of the proofs defined are short and concise and prove the way base cases behave, while others require multiple mathematical lemmas that are not part of the Coq built-in library. Given the extensive scope of this section, we will focus our discussion on a representative sample that illustrates the methodology and the steps taken in our verification process.

One particularly important property is shown in *lookup_after_update* and it concerns how **update** and **lookup** work together. Intuitively, if we update a node at a given index, then looking up that same index should yield the new value. In contrast, if we update a different node, then performing a lookup on an unaffected index must give the same result as a lookup on the original tree. Consequently, we define the proof as follows:

```
Lemma lookup_after_update {V} (t : BraunTree V) (i j : nat) (v : V) :
  IsBraun t -> lookup (update i v t) j = if (i =? j) && (i <? sizeOrg t)
    then Some v
    else lookup t j.
```

We introduce the hypothesis and remark two main cases regarding the equality between the index used by update and the index used by lookup. In order to split the goal into two subgoals we will apply the destruct tactic:

```
intro Hbraun. destruct (i =? j) eqn:Heq.
- (* Case in which i = j*)
- (* Case in which i =! j*)
```

Since there are two main cases we will work two helper lemmas, one for each case. Starting with the equality case, we define the lemma:

```
Lemma lookup_after_update_Some {V} (t : BraunTree V) (i j : nat) (v : V) :
  IsBraun t -> i = j /\ i < sizeOrg t -> lookup (update i v t) j = Some v.
```

The lemma not only uses as a prerequisite the invariant, but also the fact that the index must be within bounds. We proceed by induction on the tree t and, since in the base case the tree has size 0, the assumption that i $<$ sizeOrg t is impossible. In the proof we apply the contradiction tactic to denote this absurdity:

```
intros Hbraun. revert i j.
induction t as [| tl IHl v' tr IHr].
- simpl. intros i j. intro Hyp. destruct Hyp as [Hyp1 Hyp2].
  assert (i < 0 = False) by lia.
  rewrite H in Hyp2. contradiction.
```

In the inductive case the tree if of the form Braun tl v' r. This immediately splits into two cases according to whether i is 0 or a successor. In the case when i=0 update simply replaces the root. By doing destruct on j as well we observe two possibilities. If j is 0 the goal is solved by reflexivity but in the opposite case the hypothesis i=j is false, leading to a contradiction:

```
- intros i j Hyp. simpl. destruct i eqn:Hi.
  + simpl. destruct j.
    * reflexivity.
    * destruct Hyp. discriminate H.
```

When i is a successor, we inspect the parity of (S n). In the case when (S n) is odd, the update is applied to the left subtree. By destructing j, we either encounter a contradiction or we further destruct a boolean arising from the helper function used in the recursive update on the left subtree. The induction hypothesis IHl is applied to the left subtree. In order to do so we must show that the two conditions (the equality of indices and the size bound) hold in the recursive call. Most of the steps taken are dedicated to rewriting mathematical expressions:

```
+ simpl. rewrite Nat.sub_0_r. destruct (Nat.odd (S n)) eqn:Hodd.
      * simpl. destruct j eqn:Hj.
```

```
        ** destruct Hyp as [Hyp1 Hyp2]. discriminate Hyp1.
        ** destruct (Nat.even n0) eqn:Heven.
          *** apply IHl. inversion Hbraun; assumption.
              destruct Hyp as [Hyp1 Hyp2]. split.
            **** f_equal. assert (S n = S n0 -> n = n0) by lia. apply H in
                 Hyp1.
                 assumption.
            **** simpl in Hyp2. inversion Hbraun.
                 assert (S n < S (sizeOrg tl + sizeOrg tr) -> n < sizeOrg
                     tl + sizeOrg tr) by lia.
                 apply H5 in Hyp2; clear H5. destruct H4.
              -- (* Mathematical transformation ... *) reflexivity.
              -- (* Mathematical transformation ... *)
                 apply odd_even_div2 in H6.
                ++ rewrite Nat.double_twice in H6.
                   assert (Nat.div2 (2 * sizeOrg tl) = sizeOrg tl).
                     { rewrite <- add_n_n_twice. rewrite div2_double.
                         reflexivity. }
                   rewrite <- H7; assumption.
                ++ assert (n = n0) by lia. rewrite <- H7 in Heven.
                   rewrite <- Nat.odd_succ in Heven. rewrite <-
                       Nat.odd_spec. assumption.
                ++ rewrite Nat.double_twice.
                   assert (2 * sizeOrg tl = sizeOrg tl * 2) by lia.
                       rewrite H7.
                   apply Nat.Even_mul_r. exists 1. simpl. reflexivity.
          *** destruct Hyp. assert ( n = n0). { lia. } rewrite
              Nat.odd_succ in Hodd.
              rewrite H1 in Hodd. rewrite Hodd in Heven. discriminate
                  Heven.
```

In the second subcase (when (S n) is even), the update is applied to the right subtree. A similar approach is performed. After proving the auxiliary lemma of the first case we move to the second lemma which is defined as follows:

```
Lemma lookup_after_update_Lookup {V} (t: BraunTree V) (i j: nat) (v: V) :
  IsBraun t -> i =? j = false -> lookup (update i v t) j = lookup t j.
```

The proof again proceeds by induction on the structure of t and a case analysis on the indices. Many of the same techniques and tactics from the previous proof are reused here. In particular, we destruct the tree and then, using the definitions of update and lookup, we observe that when the update is not made on the branch where the lookup is performed, the recursive calls in the corresponding subtree remain unaltered. The final result is obtained by applying the induction hypothesis on the appropriate subtree and rewriting equalities where necessary. Combining the two cases and doing some further mathematical rewriting we finalize the main proof:

```
Lemma lookup_after_update {V} (t : BraunTree V) (i j : nat) (v : V) :
  IsBraun t -> lookup (update i v t) j = if (i =? j) && (i <? sizeOrg t)
    then Some v
    else lookup t j.
Proof.
  intro Hbraun. destruct (i =? j) eqn:Heq.
    - destruct (i <? sizeOrg t) eqn:Hsize.
      + simpl. apply lookup_after_update_Some. assumption. split.
        * rewrite Nat.eqb_eq in Heq. assumption.
        * rewrite Nat.ltb_lt in Hsize. assumption.
      + simpl. rewrite update_idempotent. reflexivity. assumption.
        rewrite Nat.ltb_nlt in Hsize.
        assert (i >= sizeOrg t <-> sizeOrg t <= i).
        { lia. }
        rewrite H. assert ( ~ i < sizeOrg t -> i >= sizeOrg t).
        { lia. }
        left. apply H0 in Hsize; assumption.
    - simpl. apply lookup_after_update_Lookup; assumption.
```

```
Qed.
```

Listing 12: The main lookup_after_update proof

The *lookup_after_update* example provides the basic approach for solving proofs with an algebraic specification. Due to space limitations, the full list of algebraic proofs is not included. However, the complete collection can be found inside the codebase's repository: [1].

3.8.2 Model-based Specification

In the case of model-based proofs we employed the function *tree_to_list* to compare our defined tree operations to the standard predefined list operations. While some proofs required minimal effort, others did not have counterpart operations available in Coq's built-in system. Thereore, we needed to define those ourselves.

```
Lemma length_merge_lists : forall (V : Type) (l r : list V),
    length (merge_lists l r) = length l + length r.
Proof.
  intros V l r.
  generalize dependent r.
  induction l as [| x xs IHxs]; intros r.
  - simpl. reflexivity.
  - destruct r as [| y ys].
    + simpl. rewrite Nat.add_0_r. reflexivity.
    + simpl. rewrite IHxs. simpl. lia.
Qed.

Lemma size_list_equiv : forall (V : Type) (t : BraunTree V),
    sizeOrg t = length (tree_to_list t).
Proof.
  intros V t.
  induction t as [| l IHl v r IHr].
  - simpl. reflexivity.
  - simpl. rewrite IHl. rewrite IHr. rewrite length_merge_lists.
      reflexivity.
Qed.
```

Listing 13: Proving size on a tree is equal to length on a list

The above theorem represents a suitable example of a trivial mode-based proof. We tackled it by applying induction on the tree pattern. A helper lemma was required in the inductive case in order to prove that the length of two lists is equal to the length of the same lists merged into one.

```
Lemma lookup_to_list_equiv : forall (V : Type) (t : BraunTree V) (i : nat),
    IsBraun t -> lookup t i = nth_error (tree_to_list t) i.
```

Listing 14: Proving lookup on a tree is equal to nth$_e$*rroronalist*

The proof of *lookup_to_list_equiv* heavily relies on several auxiliary lemmas that were implemented on the function *nth_error*, since Coq lacks extensive integrated theorems on *nth_error*. To facilitate induction, the proof generalizes over the index i. This way the induction hypothesis can be applied uniformly across all possible indices. The base case can be trivially demonstrated by destructing the index number i. In the inductive case, the tree t is of the form Braun l v r. The destruct tactic is used again on the index number i. The case in which i=0 simplifies *lookup* (*Braun l v r*) 0 directly to Some v. *Nth_error* on 0 simplifies to Some v as well, thus, the equality holds by reflexivity. In the case when i>0, the proof proceeds by analyzing the parity of the index and traversing the tree accordingly. In both cases, the proof follows a similar pattern, it simplifies the expressions and applies the induction hypothesis on the corresponding subtree. Here, the auxiliary lemmas *nth_error_merge_lists_even* and *nth_error_merge_lists_odd* are invoked in order to prove that *nth_error* on a merged list is equal to *nth_error* on the first list at a half of the index if the index is even and on the second list if the index is odd:

$$(length\, l1 = length\, l2 \vee length\, l1 = length\, l2{+}1) \rightarrow nth\_error(merge\_lists\, l1\, l2)(2{*}k) = nth\_error\, l1\, k. \tag{2}$$

$$(length\, l1 = length\, l2 \vee length\, l1 = length\, l2{+}1) \rightarrow nth\_error(merge\_lists\, l1\, l2)(2{*}k{+}1) = nth\_error\, l1\, k. \tag{3}$$

The rest of the model-based proofs implemented can be seen below :

```
(*Proving update in a Braun Tree is equal to the upd operator in a list*)
Lemma update_to_list_equiv : forall (V : Type) (t : BraunTree V) (i : nat)
    (v : V),
    IsBraun t -> tree_to_list (update i v t) = upd (tree_to_list t) i v.

(*Proving the tuple produced by removeRoot is equal to (head::tail) in a
    list*)
Lemma remove_to_list_equiv {V : Type} (t : BraunTree V) (v : V) :
    IsBraun t -> forall t', removeRoot t = Some (v, t') -> tree_to_list t
        = v :: tree_to_list t'.

(*Proving insert in a Braun Tree is equal to the ++ operator in a list*)
Lemma insert_to_list_equiv : forall (V : Type) (v : V) (t : BraunTree V),
    IsBraun t -> tree_to_list (insert v t) = tree_to_list t ++ [v].
```

Listing 15: Implemented model-based proofs

3.8.3 Invariant Maintenance

The proofs shown in this section ensure that the operations on Braun Trees preserve the shape property. For each operation one relatable lemma was approached. For the read-only operations that do not alter the content of the trees such as size or lookup no proof was developed.

```
Lemma insert_maintains_braun : forall V (v : V) (t : BraunTree V),
  IsBraun t -> IsBraun (insert v t).
Proof.
  intros V v t Hbraun. induction t as [|l IHl v' r IHr].
  - simpl. constructor; simpl; constructor. reflexivity.
  - simpl. destruct (Nat.eqb (sizeOrg l) (sizeOrg r)) eqn:Heq.
    + constructor.
      * apply IHl. inversion Hbraun; assumption.
      * inversion Hbraun; assumption.
      * simpl. rewrite size_insert_inc.
        inversion Hbraun as [|lF vF rF H0 H1 H2]. right.
        apply Nat.add_cancel_r with (p := 1). rewrite Nat.eqb_eq in Heq.
            assumption.
    + constructor.
      * inversion Hbraun. assumption.
      * inversion Hbraun. apply IHr. assumption.
      * inversion Hbraun. left. destruct H4.
        ** apply Nat.eqb_neq in Heq. contradiction.
        ** rewrite H4. rewrite size_insert_inc. reflexivity.
Qed.
```

Listing 16: Invariant Maintenance on Insert

The lemma for 'Insert' proves that if IsBraun t holds before the operation, it will also hold after the operation. The proof is structured inductively on the form of the tree. In the empty case, t is *Empty*. An insert into the Empty Tree will yield a single node tree that satisfies the invariant. In the inductive step, we rely on the size equality between the two subtrees. If they are equal, we insert into the left subtree and show that the new size difference remains acceptable. If they do not match, we insert into the right subtree show that the final shape respects the property. A key element in proving this lemma is having the $IsBraun\ t$ as a hypothesis.

```
Lemma replicate_is_braun : forall {V : Type} (x : V) (n : nat) ,
  IsBraun (replicate x n).
Proof.
  intros. induction n as [| n' IH].
  - simpl. constructor.
  - simpl. apply insert_maintains_braun. assumption.
Qed.
```

Listing 17: Replicate generates a Braun Tree

replicate_is_braun proceeds by using induction on the natural number representing the number of nodes. In the base case when n is 0, replicate will output the Braun Tree *Empty* which was already mentioned. In the inductive case, we can already assume *replicate* x n' is a valid Braun Tree. This also leads to *replicate* x S n' being a Braun Tree since we already proved that insert maintains the invariant.

```
Lemma removeRoot_maintains_braun : forall (V : Type) (t : BraunTree V) v
    t',
    IsBraun t -> removeRoot t = Some (v, t') -> IsBraun t'.
Proof.
    (...)
Qed.

Lemma remove_maintains_braun : forall (V : Type) (t : BraunTree V),
    IsBraun t -> forall t', remove t = Some t' -> IsBraun t'.
Proof.
    (...)
Qed.
```

Listing 18: Invariant Maintenance on Remove

In proving that *remove_maintains_braun* holds we needed to define a new theorem for proving that remove's helper operation, removeRoot, also holds. The proof also relies on the assumption that remove succeeds and returns a new tree t'.

```
Lemma update_maintains_braun : forall (V: Type) (t: BraunTree V) (i: nat)
    (v: V),
    IsBraun t -> IsBraun (update i v t).
Proof.
    (...)
Qed.
```

Listing 19: Invariant Maintenance on Update

Even though update does not particularly alter the structure of a Braun Tree, since it only deals with the values inside the nodes, it can still prove beneficial to show that it preserves the invariant. We again perform structural induction on the tree and easily discard the base case. We then destruct the index at which the update is performed. In the case when i is 0, the update is done on the root and thus the balance remains the same. When $i > 0$, we need to analyze each case: when update occurs in the left subtree or in the right subtree. The induction hypothesis is then applied to the respective subtree to ensure that the update operation within that subtree maintains its $IsBraun$ property.

4 Complexity Analysis

A direct consequence of the shape constraint is that operations on Braun Trees often run in $O(\log n)$ time, where n is the size of the tree. The following operations were defined satisfying this efficient complexity level:

- **insert** runs in $O(\log n)$ by recursing down the tree height, guided by the size comparison between the left and right subtrees.
- **lookup** and **update** have a similar approach, as they effectively do repeated halving down the tree based on the parity of the index.
- **remove** also runs in $O(\log n)$ time due to doing a single pass to remove and after switching the subtrees following the same process in the right one.

However, the rest of the implemented operations provide naive linear approaches that take $O(n)$ time to compute. Okasaki's more sophisticated **size** and **replicate** algorithms [3] achieve a time complexity of $O(\log n)$ by exploiting parity and the balanced structure of Braun Trees. We partially implemented these specialized algorithms but future works might require proving the equality with the linear solutions.

5 Future Work

While significant progress has been made, several tracks remain open for future research. A natural progression would be to extend the proofs to cover all of Okasaki's algorithms. This could be done by proving the equivalence between the logarithmic algorithms and the slow definitions presented in the paper. However, the first step taken into this direction would be to formulate the list to tree conversion operation.

Defining it would complement the available tree to list function and would give the opportunity of formalizing Okasaki's third algorithm, which was not covered due to its complexity and scope.

Lastly, using Coq's extraction tools, the verified operations could be used outside of the proof system. This would demonstrate the practical use of the project, by using Braun Trees in developing other data structures.

6 Results and Conclusion

The solutions presented in this project answer the questions proposed in the problem statement, by providing a fully verified framework that confirms the mathematical correctness of Braun Trees. Each operation: size, replicate, lookup, update, insert, remove and tree to list; has been defined and verified accordingly. These operations maintain the tree's invariant and perform as expected. Moreover, this thesis highlights their practicality, positioning them as reliable components within extensible arrays, priority queues, and similar data structures.

In conclusion, this project marks a significant milestone in the formal proof of data structures, specifically Braun Trees, advancing both their theoretical and practical understanding and setting the stage for further innovations. The complete implementation of the discussed proofs can be accessed at the following repository: [1].

7 Acknowledgments

I would like to express my sincere gratitude to my thesis supervisor, Dr. Dan Frumin, for his patience, consistent support and guidance throughout the entire project. His expertise into Coq and formal verification proved instrumental in completing the thesis.

References

[1] A. Filiuta, "Braun trees in coq," https://github.com/alexfiliuta/BraunTreesInCoq.

[2] W. Braun and M. Rem, "A logarithmic implementation of flexible arrays," *Memorandum MR83/4, Eindhoven University of Technology*, 1983.

[3] C. Okasaki, "Three algorithms on braun trees," *Journal of Functional Programming*, vol. 7, pp. 661 – 666, 1997. [Online]. Available: https://api.semanticscholar.org/CorpusID:34429104

[4] T. Nipkow and T. Sewell, "Proof pearl: Braun trees," in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs*, ser. CPP 2020. New York, NY, USA: Association for Computing Machinery, 2020, p. 18–31. [Online]. Available: https://doi.org/10.1145/3372885.3373834

[5] J.-C. Filliâtre and P. Letouzey, "Functors for proofs and programs," in *Programming Languages and Systems*, D. Schmidt, Ed. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 370–384.

[6] B. C. P. et al, *Software Foundations*. Electronic textbook, 2023. [Online]. Available: https://softwarefoundations.cis.upenn.edu/